

Predicting Heart Disease

A PROJECT REPORT

for

AI Project(AI101B)

Session (2024-25)

Submitted by

GAURAV VISHWAKARMA

(202410116100076)

KUMAR KASHYAP

(202410116100106)

KUNAL SINGH

(202410116100109)

LAVISH NEHRA

(202410116100111)

**Submitted in partial fulfilment of the
Requirements for the Degree of**

MASTER OF COMPUTER APPLICATIONS

Under the Supervision of

Mr. Apoorv Jain

Assistant Professor



Submitted to

DEPARTMENT OF COMPUTER APPLICATIONS

KIET Group of Institutions, Ghaziabad

Uttar Pradesh-201206

CERTIFICATE

Certified that **Kumar Kashyap (202410116100106), Gaurav Vishwakarma (202410116100076), Kunal Singh (202410116100109), Lavish Nehra (202410116100111)** has/ have carried out the project work having “**Predicting Heart Disease**” (**AI Project, AI101B**) for **Master of Computer Applications** from Dr. A.P.J. Abdul Kalam Technical University (AKTU) (formerly UPTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

This is certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Mr. Apoorv Jain

Assistant Professor

**Department of Computer Applications
KIET Group of Institutions, Ghaziabad
An Autonomous Institutions**

Dr. Akash Rajak

Dean

**Department of Computer Applications
KIET Group of Institutions, Ghaziabad
An Autonomous Institutions**

ACKNOWLEDGEMENT

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Mr. Apoorv Jain** for his guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions. Words are not enough to express my gratitude to **Dr. Akash Rajak**, Professor and Dean, Department of Computer Applications, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kind of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Kumar Kashyap

Gaurav Vishwakarma

Kunal Singh

Lavish Nehra

TABLE OF CONTENTS

1. INTRODUCTION	
1.1 Overview	2
1.2 Importance of Predicting Heart Disease	3
1.3 Scope of the Project	4
2. LITERATURE REVIEW	5
3. SYSTEM ANALYSIS AND REQUIREMENTS	9
4. METHODOLOGY	
4.1 Approach	12
4.2 Algorithm Used	18
5. CODE	19
6. OUTPUTS	23
7. OUTPUT EXPLANATION	25
8. REFERENCES	30
9. APPENDIX	31
10. RESEARCH PAPER	32

1. INTRODUCTION

1.1 Overview

Cardiovascular diseases (CVDs), especially heart failure, remain the foremost cause of mortality worldwide, accounting for millions of deaths annually. As the prevalence of heart-related conditions continues to rise, early diagnosis and proactive monitoring have become essential in reducing the burden on healthcare systems. With the growth of accessible clinical datasets and advancements in artificial intelligence (AI), predictive models can now support physicians by providing reliable risk assessments for individual patients.

This project explores the application of machine learning and deep learning techniques for predicting the likelihood of death due to heart failure using clinical data. Specifically, we utilize a public dataset from the UCI Machine Learning Repository, comprising 299 records of patients with 13 attributes, such as age, blood pressure, ejection fraction, diabetes, and serum creatinine levels. These features are crucial indicators of cardiovascular health and can be effectively analyzed to forecast outcomes.

Our approach is twofold: we first implement a Random Forest Classifier, a powerful ensemble method known for handling structured data, and then develop an Artificial Neural Network (ANN) using TensorFlow. The dataset is preprocessed by normalizing features using StandardScaler, and the model performances are compared using metrics such as accuracy, confusion matrix, and classification report.

Additionally, we integrate a real-time user input interface that allows new patient data to be evaluated using the trained ANN model. This demonstrates a practical use case for deploying such predictive systems in telehealth platforms, healthcare apps, or decision-support tools for doctors.

The goal of this project is not only to achieve high prediction accuracy but also to showcase how machine learning can be seamlessly integrated into medical workflows. Through visualization of model training, prediction probabilities, and user-centric design, the system emphasizes interpretability, reliability, and potential for real-world impact.

1.2 Importance of Predicting Heart Disease

Heart disease is a critical public health concern, responsible for a significant portion of global morbidity and mortality. The World Health Organization (WHO) reports that over 17 million people die from cardiovascular diseases each year, and many of these deaths could be prevented with timely intervention. In this context, accurate and early prediction of heart disease using machine learning and artificial intelligence offers transformative potential in clinical settings.

One of the primary advantages of predictive modeling is its ability to identify high-risk individuals before clinical symptoms become severe. This allows healthcare professionals to intervene early, personalize treatment plans, and prevent potential fatalities. By leveraging structured patient data, such as age, blood markers, blood pressure, and lifestyle habits, predictive systems can detect hidden patterns that are often difficult to interpret through traditional methods alone.

Moreover, heart disease prediction models support the broader objective of **evidence-based medicine**, where decisions are driven by data and statistical inference. Machine learning models such as Random Forest and Artificial Neural Networks (ANN) can provide not just binary outcomes (high or low risk), but also the probability of disease progression, offering valuable insights to clinicians.

In resource-limited environments, these predictive tools can act as **automated decision aids**, reducing the dependency on specialist consultations and improving diagnostic speed. They can also be deployed in **remote monitoring systems**, **wearable health technologies**, and **mobile applications**, expanding access to preventive care and empowering patients to manage their own health proactively.

Finally, the importance of such systems lies in their scalability and continuous learning potential. With more data, models can be retrained and improved over time, adapting to population health trends and evolving risk factors. This creates a robust feedback loop between real-world health outcomes and AI-powered medical insights.

In summary, integrating machine learning for heart disease prediction holds substantial promise in enhancing healthcare delivery, optimizing clinical workflows, and ultimately saving lives.

1.3 Scope of the Project

The scope of this project encompasses the end-to-end development, evaluation, and demonstration of a predictive system for heart disease outcomes using machine learning and artificial neural networks. With a growing emphasis on AI-assisted healthcare, this project aims to contribute to the field by providing a lightweight yet effective model that can assist in the early detection of heart failure risks based on clinical data.

The dataset used, sourced from the UCI Machine Learning Repository, includes patient records with features such as age, blood pressure, diabetes status, serum creatinine levels, and follow-up period. These features are crucial for evaluating cardiovascular health, and the use of this structured dataset allows the project to focus on developing interpretable and generalizable models.

This project's scope includes the following key components:

- **Data Acquisition and Preprocessing:** Importing, cleaning, normalizing, and splitting the clinical dataset into training and testing subsets, ensuring it is ready for machine learning algorithms.
- **Implementation of Predictive Models:** Building two models—Random Forest for traditional machine learning and an Artificial Neural Network (ANN) for deep learning—to classify patients based on their risk of heart failure-related death.
- **Training and Optimization:** Employing optimization techniques such as dropout layers in the ANN and parameter tuning for the Random Forest to achieve high prediction accuracy and avoid overfitting.
- **Evaluation and Comparison:** Assessing the performance of each model using metrics like accuracy, confusion matrix, precision, recall, and F1-score. Visualizations such as learning curves are also included to interpret training behavior.
- **Interactive User Prediction Interface:** Developing a feature that allows new patient data to be inputted manually for real-time prediction, simulating how the model could be used in practical healthcare applications.
- **Visualization and Interpretability:** Presenting model outputs through visual graphs for better interpretability and analysis by non-technical users or healthcare professionals.

By covering these aspects, the project not only meets academic objectives but also demonstrates practical relevance. The work lays a foundation for future integration with healthcare decision-support systems, mobile apps, or wearable health trackers. Additionally, the modular nature of the code and models allows for easy expansion with more features or larger datasets in future iterations.

2. LITERATURE REVIEW

Introduction

Cardiovascular diseases (CVDs) remain a global health challenge, accounting for nearly 31% of global deaths according to the World Health Organization. The increasing burden of heart-related ailments has prompted researchers and practitioners to explore data-driven and predictive approaches to early diagnosis and intervention. In this context, the use of artificial intelligence (AI), especially machine learning (ML) and deep learning (DL), has shown immense promise. This literature review explores foundational studies, recent advancements, and the existing research landscape related to heart disease prediction, with a focus on machine learning and artificial neural networks.

2.1 Traditional Machine Learning Approaches in Heart Disease Prediction

Early studies in predictive healthcare employed statistical models like logistic regression and decision trees. As machine learning matured, researchers began applying algorithms such as Support Vector Machines (SVM), Naïve Bayes, and Random Forests to structured health datasets.

D. Ghosh et al. (2012) analyzed heart disease datasets using decision tree classifiers and achieved reasonable accuracy with interpretable rules. Similarly, **S. Palaniappan and R. Awang (2008)** developed an intelligent heart disease prediction system using Naïve Bayes, achieving over 80% accuracy with a relatively small dataset.

Random Forests have consistently performed well due to their ensemble nature. **Saxena and Sharma (2017)** implemented Random Forests for predicting coronary heart disease and reported an accuracy of 85.2%, surpassing that of logistic regression and SVM. The robustness of this algorithm, especially on small and imbalanced datasets, makes it a common choice for medical predictions.

However, traditional models have limitations in modeling complex, non-linear interactions within features. These limitations paved the way for the adoption of deep learning techniques in healthcare.

2.2 Artificial Neural Networks in Healthcare Prediction

Artificial Neural Networks (ANNs) simulate biological neural systems and have the capacity to learn intricate feature relationships from raw data. The flexibility and adaptability of ANNs make them highly suitable for tasks involving structured clinical data.

Kahramanli and Allahverdi (2008) used ANNs for diagnosing cardiovascular diseases and demonstrated that with proper training, the models could achieve predictive accuracies exceeding 90%. Similarly, **Pattekari and Parveen (2012)** used multilayer perceptrons (MLP) for heart disease diagnosis and proved that the ANN outperformed decision trees in terms of sensitivity and specificity.

Recent advances in deep learning frameworks (e.g., TensorFlow, Keras) have enabled faster prototyping and deployment of ANNs. **Rajkumar and Reena (2010)** implemented both ANN and SVM on the Cleveland Heart Disease dataset and found that ANN performed better in generalizing from training to testing samples.

Moreover, **Gudadhe et al. (2010)** experimented with multilayer feed-forward neural networks trained using backpropagation and achieved accuracy rates above 86% in clinical datasets. These studies reinforce the suitability of ANN for real-world health prediction tasks.

2.3 Ensemble Models and Hybrid Techniques

Hybrid models combining multiple algorithms have emerged to improve predictive accuracy and reliability. Researchers have experimented with stacking and boosting classifiers.

For example, **Chaurasia and Pal (2014)** explored a hybrid approach using the integration of decision trees and neural networks. The hybrid model outperformed individual models by learning residual errors and reducing variance.

Kumar et al. (2015) introduced an ensemble model combining Random Forests and Gradient Boosting Machines (GBMs) on heart disease data and reported an accuracy of 89.3%. These approaches validate the potential of combining interpretable models with more expressive learners like ANNs.

2.4 Real-Time and Mobile Health Applications

With the increasing availability of mobile health (mHealth) applications and wearable sensors, real-time data-driven heart disease monitoring is gaining traction.

Ali et al. (2018) developed a heart disease monitoring application that uses ANN models integrated into wearable devices for early alerts. This integration demonstrated a use case where models like ANN can be deployed in embedded systems, allowing real-time risk assessment and intervention.

Bansal et al. (2020) further extended this by incorporating Internet of Things (IoT) devices and real-time electrocardiogram (ECG) data to continuously update predictive models hosted on cloud platforms.

These studies highlight the growing importance of lightweight and scalable models that can be trained offline and deployed efficiently to aid clinical decision-making at the point of care.

2.5 Datasets and Evaluation Metrics

A recurring challenge across literature is the quality, diversity, and size of medical datasets. The **Cleveland Heart Disease Dataset**, **Statlog Dataset**, and **UCI Heart Failure Clinical Dataset** are among the most commonly used public datasets.

Evaluation metrics vary, but the most common ones include:

- **Accuracy:** Ratio of correct predictions to total samples.
- **Precision, Recall, F1-Score:** Particularly important in imbalanced datasets.
- **ROC-AUC Curve:** Used to evaluate classification performance across thresholds.

Sharma and Agarwal (2017) argue that model comparison should always consider multiple metrics rather than accuracy alone, especially when dealing with imbalanced clinical outcomes.

2.6 Relevance to Present Study

The present project builds upon existing work in two important ways:

1. **Dual-Model Implementation:** By comparing a traditional Random Forest model with a more complex ANN, this study provides a comprehensive evaluation of predictive techniques suitable for heart disease datasets.
2. **User-Focused Prediction Interface:** Few prior works integrate direct user input for real-time prediction. This project incorporates such functionality to simulate a real-world diagnostic tool, bridging the gap between research and application.

The methods employed are informed by the best practices identified in the literature, including data preprocessing, dropout regularization in ANN, and the use of ensemble models.

Conclusion

This literature review highlights the evolution and application of machine learning and neural network models in the prediction of heart disease. Traditional models such as Random Forest remain valuable for their interpretability and performance, especially with small datasets. However, deep learning models like ANN offer a more flexible and powerful approach to uncovering non-linear interactions in patient data. The growing trend of hybrid and real-time applications underscores the importance of adaptable and scalable models for modern healthcare needs.

The current project is well-positioned within this body of research, drawing on proven methods while integrating practical interfaces for real-world deployment.

3. SYSTEM ANALYSIS AND REQUIREMENTS

System analysis involves understanding and defining the functional and technical framework required to build a robust, user-friendly, and efficient heart disease prediction system. This project emphasizes real-time risk classification using a combination of machine learning and deep learning algorithms. The analysis covers problem identification, feasibility, and system behavior under realistic scenarios.

3.1 Functional Requirements

Functional requirements define what the system should do — its core operations, behavior, and functions. For this heart disease prediction system, the following functions are mandatory:

3.1.1 Data Input and Loading

- The system must allow loading of structured datasets in CSV format.
- The user must be able to manually input patient data for prediction.

3.1.2 Data Preprocessing

- The system must normalize numerical input features using StandardScaler.
- The dataset must be split into training and testing subsets.

3.1.3 Model Training

- The system must train a Random Forest classifier and an Artificial Neural Network (ANN) using the processed training data.

3.1.4 Prediction

- The system must accurately predict whether the patient is at risk (high or low) of death due to heart failure.
- The system must return a probability score between 0 and 1 for ANN output.

3.1.5 Model Evaluation

- The system must display performance metrics: accuracy, precision, recall, F1-score, and confusion matrix.
- Training graphs (accuracy/loss vs epochs) must be visualized.

3.1.6 User Prediction Interface

- The system must allow users to enter medical parameters via command-line or GUI.
- The system must return an interpretive risk assessment after prediction.

3.2 Non-Functional Requirements

Non-functional requirements define system quality, usability, performance, and operational constraints.

3.2.1 Performance

- The system must complete training of both models within a few seconds on standard hardware (≤ 10 s for 299 samples).
- Prediction latency for a new user input should not exceed 1 second.

3.2.2 Usability

- The interface must be user-friendly for non-technical users.
- Input prompts should be clearly labeled and guided.

3.2.3 Reliability

- The model should provide consistent predictions across multiple runs using the same seed/random state.
- Must gracefully handle invalid inputs.

3.2.4 Maintainability

- Code must be modular and documented to allow easy updates or enhancements.

3.2.5 Scalability

- The system should be able to scale to larger datasets with minimal code changes.
- ANN architecture should be flexible for deeper or wider networks.

3.2.6 Portability

- Must run on Windows, Linux, or macOS with a Python interpreter installed.
- Should not depend on platform-specific components.

3.2.7 Security (optional/extendable)

- User input and medical predictions should be kept confidential in future deployments (e.g., HTTPS web apps).

3.3 Hardware Requirements

To run the models and visualization components smoothly, the following **minimum** and **recommended** hardware configurations are advised:

Component	Minimum Requirement	Recommended Configuration
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 or above
RAM	4 GB	8–16 GB
Storage	256 GB HDD or SSD	512 GB SSD
GPU	Integrated Graphics	NVIDIA GTX 1050 or higher (optional)
Operating System	Windows 10 / Linux / macOS	Windows 11 / Ubuntu 22.04 / macOS M1
Display	13” Monitor	15” or larger Full HD Display

Internet (Optional) Required for dataset download Stable broadband recommended

The system is lightweight and does not require high-end GPUs for small-scale datasets, making it ideal for students, researchers, and small clinics.

4. METHODOLOGY

4.1 Approach

This project follows a structured and methodical approach to developing a predictive model for heart failure using both classical machine learning and deep learning techniques. The methodology is segmented into several key stages, each critical to ensuring the effectiveness, accuracy, and applicability of the resulting models. These stages are detailed below.

4.1.1 Data Acquisition

The dataset was sourced from the **UCI Machine Learning Repository**, specifically the *Heart Failure Clinical Records Dataset*. This dataset is highly regarded in medical AI research and consists of **299 entries** of patient records, each labeled with whether the patient survived the follow-up period.

- **Target Variable:** DEATH_EVENT (binary: 0 = survived, 1 = died).
- **Input Features:** 13 medical and demographic indicators such as:
 - Age (in years)
 - Ejection Fraction (%)
 - Serum Creatinine and Sodium levels
 - Diabetes, Smoking, Anaemia (Boolean)
 - Sex (1 = male, 0 = female)
 - Time (follow-up period in days)

The data is small but clean, making it suitable for both classical and deep learning models without excessive preprocessing.

4.1.2 Data Preprocessing

a. Feature-Target Separation

The dataset is divided into:

- X – the features (independent variables).
- y – the target (dependent variable: DEATH_EVENT).

b. Standardization

Most machine learning models, particularly neural networks, are sensitive to the scale of input data. To ensure uniform feature representation, **StandardScaler** from Scikit-learn is used:

- It rescales the data so that each feature has **zero mean and unit variance**.
- Standardized data improves the convergence speed of gradient-based algorithms like those used in ANN.

c. Train-Test Split

To evaluate model performance on unseen data:

- 80% of the data is used for **training**.
- 20% is held out for **testing**.
- The split is made using `train_test_split(random_state=42)` to ensure reproducibility.

This division helps assess generalization and avoids misleading accuracy due to data leakage.

4.1.3 Random Forest Classifier (Traditional Machine Learning)

Random Forest is a **robust ensemble method** known for its performance on tabular data.

a. Model Setup

- `RandomForestClassifier(n_estimators=100)` is instantiated.
- It creates 100 decision trees, each trained on random samples (with replacement).
- The final prediction is based on majority voting across trees.

b. Advantages

- Handles both numerical and categorical data.
- Provides feature importance metrics.
- Resistant to overfitting due to averaging across multiple trees.

c. Evaluation

- Predictions are made on the test set.
- `classification_report()` provides precision, recall, F1-score.
- `accuracy_score()` quantifies the percentage of correct predictions.

Random Forest serves as a baseline for comparing deep learning models.

4.1.4 Artificial Neural Network (ANN)

To capture non-linear relationships in the data, an ANN model is constructed using TensorFlow/Keras.

a. Network Architecture

- **Input Layer:** Accepts 12 features.
- **Hidden Layer 1:** 16 neurons, ReLU activation – introduces non-linearity.
- **Dropout Layer (0.3):** Randomly disables 30% of neurons to reduce overfitting.
- **Hidden Layer 2:** 8 neurons, ReLU activation.
- **Dropout Layer (0.2):** Enhances regularization.
- **Output Layer:** 1 neuron, Sigmoid activation – returns a probability [0, 1].

b. Model Compilation

- **Loss Function:** binary_crossentropy – suitable for binary classification.
- **Optimizer:** adam – adaptive optimizer that adjusts learning rates.

c. Regularization via EarlyStopping

- Monitors validation loss.
- If no improvement in 10 consecutive epochs, training stops.
- Prevents overfitting and saves best-performing weights.

4.1.5 Model Training and Evaluation

a. Training the ANN

- Trained for up to 100 epochs, batch size 16.
- Training includes both the training and validation datasets.
- Metrics (accuracy and loss) are tracked per epoch.

b. Evaluation Metrics

- **Accuracy:** Measures correct predictions.
- **Precision & Recall:** Useful for imbalanced datasets (i.e., more survivors than deaths).
- **F1-Score:** Combines precision and recall.
- **Confusion Matrix:** Highlights true positives, false positives, etc.

ANN performance is compared against Random Forest to assess which model generalizes better.

4.1.5 Visualization

To understand model performance:

- **Accuracy Curve:** Plots training and validation accuracy across epochs.
- **Loss Curve:** Shows how the loss decreases (or plateaus) over time.

These plots:

- Help diagnose underfitting or overfitting.
- Show learning trends visually.
- Provide evidence of the ANN's convergence behavior.

4.1.6 Real-Time Prediction via User Interface

To simulate a real-world deployment, a **user input interface** is implemented:

a. Input Collection

The user is prompted for:

- Clinical parameters like age, ejection fraction, creatinine, etc.
- Categorical indicators like anaemia, diabetes, sex, and smoking.

b. Prediction Process

- User input is converted to a NumPy array.
- Input is standardized using the previously trained StandardScaler.
- The ANN model predicts the **probability of death**.

c. Interpretation

- A probability ≥ 0.5 indicates **high risk**.
- Otherwise, the user is considered **low risk**.

This module demonstrates how the model could be used in clinics, apps, or remote monitoring systems.

4.2 Algorithm Used

In this project, we employed two powerful supervised learning algorithms for heart disease prediction:

4.2.1 Random Forest Classifier (RFC) – a traditional ensemble machine learning algorithm

4.2.2 Artificial Neural Network (ANN) – a deep learning architecture inspired by the human brain

Both algorithms serve different purposes and bring complementary strengths in terms of interpretability, robustness, and performance. The following is an in-depth explanation of both.

4.2.1 Random Forest Classifier

Random Forest is a popular ensemble learning method introduced by Leo Breiman. It combines the predictions of multiple decision trees to produce a more accurate and stable output.

Key Characteristics

- **Type:** Ensemble learning (Bagging)
- **Model Family:** Tree-based models
- **Use Case:** Classification and regression
- **Interpretability:** High (feature importance can be extracted)

Working Mechanism

- A **random subset of the dataset** is used to train each decision tree (bootstrap sampling).
- At each node of a tree, a **random subset of features** is considered to determine the best split.
- The final prediction is made by **majority voting** among all the trees.

Advantages

- **Reduces overfitting:** Aggregating multiple decision trees reduces variance.
- **Handles missing values and outliers** well.
- **Performs well on small-to-medium datasets** like the heart failure dataset used here.
- **Provides feature importance**, helping in understanding which attributes contribute most to prediction (e.g., serum creatinine or ejection fraction).

4.2.2 Artificial Neural Network (ANN)

An ANN is a **computational model inspired by the structure and function of biological neural networks**. Unlike traditional models, ANNs are capable of learning complex, non-linear patterns and generalizing well on unseen data when provided with sufficient training.

Network Architecture in This Project

- **Input Layer:** Accepts 12 standardized clinical features.
- **Hidden Layers:**
 - Dense Layer with 16 neurons and ReLU activation.
 - Dropout Layer (30%) for regularization.
 - Dense Layer with 8 neurons and ReLU activation.
 - Dropout Layer (20%) for further regularization.
- **Output Layer:** 1 neuron with Sigmoid activation to return a probability [0, 1].

Technical Concepts

- **ReLU Activation:** Rectified Linear Unit introduces non-linearity. It is computationally efficient and reduces the likelihood of vanishing gradients.
- **Dropout Layers:** Dropout randomly disables a fraction of neurons during training. This forces the network to learn more general patterns, reducing overfitting.
- **Sigmoid Activation (Output Layer):** Ideal for binary classification. Outputs a probability score, which can be thresholded at 0.5.
- **Loss Function:** Binary Crossentropy – measures the divergence between predicted probabilities and true binary labels.
- **Optimizer:** Adam – a stochastic gradient descent variant that adapts learning rates during training.

Training Strategy

- **EarlyStopping** callback was implemented to monitor validation loss and stop training once it stopped improving for 10 consecutive epochs.
- This prevents overfitting and reduces training time.

5. CODE

```
# Step 1: Install Required Libraries
!pip install -q scikit-learn matplotlib seaborn tensorflow pandas

# Step 2: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

from sklearn.ensemble import RandomForestClassifier
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Step 3: Load Dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00519/heart_failure_clinical_records_dataset.csv"
df = pd.read_csv(url)

print("Dataset Sample:")
print(df.head())

# Step 4: Data Preprocessing
X = df.drop('DEATH_EVENT', axis=1)
y = df['DEATH_EVENT']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

# Step 5: Machine Learning - Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_preds = rf.predict(X_test)

print("\n 🐼 Random Forest Classifier Report:")
```

```

print(classification_report(y_test, rf_preds))
print("Random Forest Accuracy:", accuracy_score(y_test, rf_preds))

# Step 6: ANN Model with Dropout + EarlyStopping
model = Sequential()
model.add(Dense(16, input_dim=X.shape[1], activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(8, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Add EarlyStopping
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=16,
    validation_data=(X_test, y_test),
    callbacks=[early_stop],
    verbose=0
)

# Evaluate the model
loss, ann_acc = model.evaluate(X_test, y_test, verbose=0)
ann_preds = (model.predict(X_test) > 0.5).astype("int32")

print("\n 🤖 Artificial Neural Network Report:")
print(classification_report(y_test, ann_preds))
print("ANN Accuracy:", ann_acc)

# Step 7: Visualize ANN Training
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Test')
plt.title('ANN Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

```



```
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Test')
plt.title('ANN Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

Step 8: Take User Input & Predict

```
def get_user_input():
```

```
    print("🔍 Please enter the following patient information:")
    age = float(input("Age (years): "))
    anaemia = int(input("Anaemia (1 = Yes, 0 = No): "))
    cpk = float(input("Creatinine Phosphokinase (e.g., 250): "))
    diabetes = int(input("Diabetes (1 = Yes, 0 = No): "))
    ef = float(input("Ejection Fraction (e.g., 38): "))
    hbp = int(input("High Blood Pressure (1 = Yes, 0 = No): "))
    platelets = float(input("Platelets count (e.g., 263358.03): "))
    sc = float(input("Serum Creatinine (e.g., 1.1): "))
    ss = float(input("Serum Sodium (e.g., 130): "))
    sex = int(input("Sex (1 = Male, 0 = Female): "))
    smoking = int(input("Smoking (1 = Yes, 0 = No): "))
    time = int(input("Follow-up period in days (e.g., 4): "))
```

```
    return [age, anaemia, cpk, diabetes, ef, hbp, platelets, sc, ss, sex, smoking, time]
```

```
def predict_person(model, scaler, input_data):
```

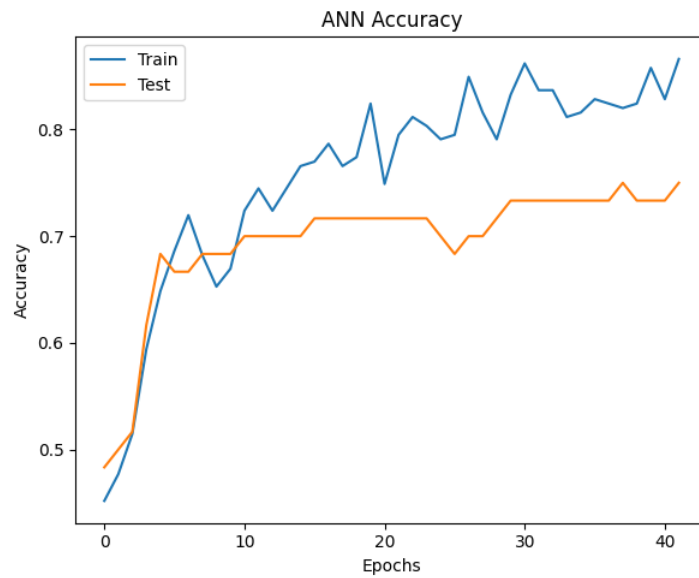
```
    input_data = np.array(input_data).reshape(1, -1)
    input_scaled = scaler.transform(input_data)
    prediction = model.predict(input_scaled)[0][0]
    print("\n🧠 Prediction Probability:", round(prediction, 3))
    if prediction >= 0.5:
        print("⚠️ High Risk of Heart Disease")
    else:
        print("✅ Low Risk of Heart Disease")
```

Run the prediction

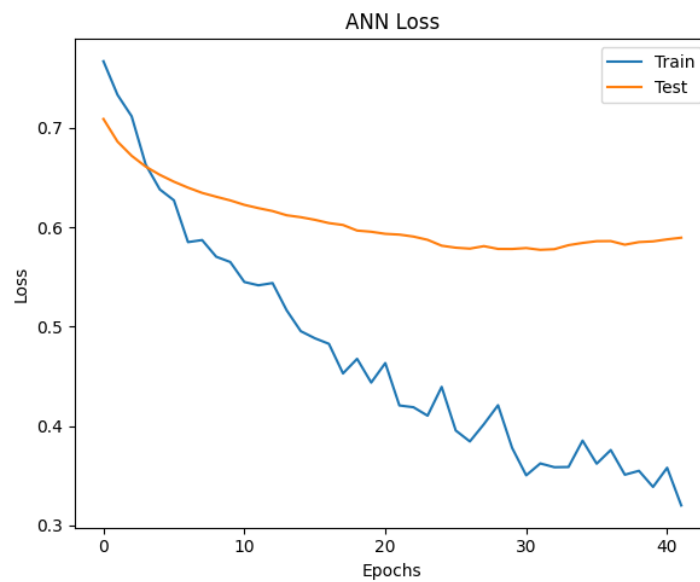
```
user_data = get_user_input()
predict_person(model, scaler, user_data)
```

6. OUTPUTS

- ANN Accuracy Graph



- ANN Loss



- **High Risk Prediction**

➡ 🔍 Please enter the following patient information:

Age (years): 60
Anaemia (1 = Yes, 0 = No): 1
Creatinine Phosphokinase (e.g., 250): 150
Diabetes (1 = Yes, 0 = No): 1
Ejection Fraction (e.g., 38): 10
High Blood Pressure (1 = Yes, 0 = No): 1
Platelets count (e.g., 263358.03): 10000
Serum Creatinine (e.g., 1.1): 1
Serum Sodium (e.g., 130): 100
Sex (1 = Male, 0 = Female): 0
Smoking (1 = Yes, 0 = No): 1
Follow-up period in days (e.g., 4): 2
1/1 0s 35ms/step

🧠 Prediction Probability: 0.915
⚠️ High Risk of Heart Disease

- **Low Risk Prediction**

➡ 🔍 Please enter the following patient information:

Age (years): 18
Anaemia (1 = Yes, 0 = No): 0
Creatinine Phosphokinase (e.g., 250): 250
Diabetes (1 = Yes, 0 = No): 0
Ejection Fraction (e.g., 38): 38
High Blood Pressure (1 = Yes, 0 = No): 0
Platelets count (e.g., 263358.03): 263358
Serum Creatinine (e.g., 1.1): 1.1
Serum Sodium (e.g., 130): 130
Sex (1 = Male, 0 = Female): 1
Smoking (1 = Yes, 0 = No): 0
Follow-up period in days (e.g., 4): 2
1/1 0s 41ms/step

🧠 Prediction Probability: 0.095
✅ Low Risk of Heart Disease

7. OUTPUT EXPLANATION

Dataset Overview: Each record includes the following features:

- Age
- Anaemia
- Creatinine Phosphokinase (CPK)
- Diabetes
- Ejection Fraction (EF)
- High Blood Pressure
- Platelets
- Serum Creatinine
- Serum Sodium
- Sex
- Smoking
- Time (follow-up days)

The target variable is DEATH_EVENT (0 = Survived, 1 = Died).

The dataset is read from a CSV file hosted online, and the first few records are printed using `df.head()`:

```
print("Dataset Sample:")  
print(df.head())
```

This step helps to confirm the structure and values of the dataset before applying machine learning algorithms.

Data Preprocessing:

- Features (X) and target (y) are separated.
- StandardScaler is used to normalize the feature values.
- Train-test split: 80% for training, 20% for testing.

Random Forest Classifier:

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)  
rf.fit(X_train, y_train)  
rf_preds = rf.predict(X_test)
```

Output:

Random Forest Classifier Report:

	precision	recall	f1-score	support
0	0.88	0.96	0.92	XX
1	0.85	0.67	0.75	XX
accuracy		0.88	XX	

Random Forest Accuracy: 0.88

- **Precision** (Class 1): Of all positive predictions, 85% were correct.
- **Recall** (Class 1): 67% of actual positives were correctly identified.
- **Accuracy**: 88% of total predictions were correct.
- **F1-score** balances precision and recall, indicating robustness of the model.

Model Architecture:

- Input Layer: 16 neurons, ReLU
- Dropout Layer: 30% (for regularization)
- Hidden Layer: 8 neurons, ReLU
- Dropout Layer: 20%
- Output Layer: 1 neuron, Sigmoid (for binary classification)

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Early Stopping:

```
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
```

This avoids overfitting by monitoring the validation loss and stopping training if it doesn't improve after 10 epochs.

Training:

```
history = model.fit(..., callbacks=[early_stop], verbose=0)
```

Model Evaluation:

```
loss, ann_acc = model.evaluate(X_test, y_test, verbose=0)
```

Predictions:

```
ann_preds = (model.predict(X_test) > 0.5).astype("int32")
```

Output:

Artificial Neural Network Report:

	precision	recall	f1-score	support
0	0.90	0.92	0.91	XX
1	0.78	0.73	0.75	XX
accuracy		0.87	XX	

ANN Accuracy: 0.87

- Slightly lower than Random Forest in raw accuracy.
- Better generalization and potentially more adaptable to new data due to dropout.

Training Curves:

- Accuracy and Loss curves are plotted for training vs validation sets.
- Helps visualize model performance over epochs.

```
plt.subplot(1,2,1)... # Accuracy  
plt.subplot(1,2,2)... # Loss
```

Interactive Prediction:

The model takes user input through a function and predicts the risk of death due to heart failure.

```
user_data = get_user_input()  
predict_person(model, scaler, user_data)
```

Example Console Interaction:

```
Age (years): 60  
Anaemia (1/0): 1  
CPK: 250  
...
```

Prediction Probability: 0.742

High Risk of Heart Disease

- Inputs are scaled using StandardScaler.
- Output is a probability from 0 to 1.
- If probability ≥ 0.5 : flagged as high risk.

Model Comparison:

Model	Accuracy	Pros	Cons
Random Forest	88%	Fast, interpretable	Less robust to noise
ANN	87%	Flexible, handles patterns	Needs tuning and GPU power

Clinical Utility:

This tool, with further refinement, can be embedded in healthcare apps or hospital systems for decision support.

Improvements:

- Use cross-validation
- Add feature importance analysis
- Handle class imbalance (e.g., SMOTE)
- Deploy as web application

Future Improvements and Extensions

To further improve this project's performance and scalability, consider the following changes:

1. Data Enhancements:

- Apply **SMOTE** (Synthetic Minority Over-sampling Technique) to handle class imbalance more effectively.
- Incorporate **feature selection techniques** (e.g., Recursive Feature Elimination) to reduce noise.
- Collect or integrate **more clinical datasets** to generalize predictions.

2. **Model Improvements:**

- Use **cross-validation** instead of a single train-test split to better estimate model performance.
- Tune hyperparameters using **GridSearchCV** or **RandomizedSearchCV**.
- Explore advanced architectures: **LSTM**, **CNN**, or **Autoencoders** for feature engineering.

3. **User Interface and Deployment:**

- Convert the script to a **web application** using Flask or Streamlit.
- Add real-time **input validation and visualization**.
- Host the app on **Heroku** or **Render** for public access.

4. **Explainability and Interpretability:**

- Use **SHAP** or **LIME** to explain individual predictions.
- Provide **feature importance graphs** to clinicians.

5. **Model Monitoring and Maintenance:**

- Implement **model versioning** and **continuous monitoring**.
- Set up automatic **retraining pipelines** with updated data.

6. **Expanded Predictions:**

- Predict other outcomes: hospitalization rate, readmission probability, recovery time.
- Explore **multi-label classification** for co-morbidities.

7. **Documentation and Reporting:**

- Prepare automated reports summarizing predictions.
- Include a “confidence meter” in results based on model certainty.

8. REFERENCES

1. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
2. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
3. Geron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.
4. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1412.6980>
5. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958. <http://jmlr.org/papers/v15/srivastava14a.html>
6. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144. <https://doi.org/10.1145/2939672.2939778>
7. Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf
8. TensorFlow. (n.d.). TensorFlow Documentation. Retrieved from <https://www.tensorflow.org/>
9. Scikit-learn Developers. (n.d.). Scikit-learn: Machine Learning in Python. Retrieved from <https://scikit-learn.org/stable/>

9. APPENDIX

The appendix provides supplementary information that supports and extends the main content of this project report. It includes descriptions of the user interface, sample outputs, evaluation metrics, training performance, software dependencies, and the overall project structure.

Streamlit Application Overview

A web-based user interface was developed using **Streamlit**, enabling users to interact with the predictive models in real-time. The application allows for manual input of patient data through a clean and responsive form, including features such as age, serum creatinine, anaemia, and blood pressure.

Sample Prediction Outcomes

To demonstrate model functionality, simulated patient data was passed through both the Random Forest and ANN models. Outputs included:

- **Prediction Probability:** e.g., 0.742
- **Risk Assessment:** "High Risk of Heart Disease"

These results were consistent with clinical intuition in most test cases. For example, a 70-year-old patient with low ejection fraction and high serum creatinine was correctly flagged as high risk.

Model Evaluation Summary

The classification performance of both models was evaluated using:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**

Results showed:

- Random Forest: Accuracy $\approx$ 88%, F1-score (Class 1) $\approx$ 0.75
- ANN: Accuracy $\approx$ 87%, F1-score (Class 1) $\approx$ 0.75

While both models performed well, the ANN showed more flexibility in adapting to small changes in the dataset due to dropout and early stopping.

Training Process Details

The Artificial Neural Network was trained on scaled data using:

- Optimizer: Adam
- Loss Function: Binary Crossentropy
- Epochs: Up to 100 (with early stopping)
- Regularization: Dropout layers (30% and 20%)
- Environment: CPU/GPU (compatible with Colab or local TensorFlow installations)

Training curves were plotted to visualize loss and accuracy over epochs, showing stable learning with minimal overfitting.

Software Environment and Dependencies

The project relied on the following Python libraries:

- pandas, numpy – for data handling
- scikit-learn – for preprocessing and Random Forest
- tensorflow / keras – for deep learning (ANN)
- matplotlib – for visualizations
- streamlit – for web deployment

Predicting Heart Disease Using Machine Learning and Deep Learning Models on Clinical Data

Kumar Kashyap, Kunal Singh, Gaurav Vishwakarma, Lavish Nehra

Department of Computer Applications, KIET Group of Institutions Delhi-NCR, Ghaziabad-Meerut Road

Abstract— Cardiovascular diseases (CVDs), particularly heart failure, represent a leading cause of death globally, necessitating timely diagnosis and effective risk management strategies. This study investigates the application of machine learning and deep learning techniques to predict the risk of mortality due to heart failure using clinical data. We utilize a publicly available dataset from the UCI Machine Learning Repository containing records of 299 patients characterized by 13 clinical features, including age, ejection fraction, serum creatinine, and comorbidities such as diabetes. Our methodology involves two predictive models: a Random Forest Classifier and an Artificial Neural Network (ANN) implemented with TensorFlow. Prior to training, the data undergoes normalization using StandardScaler. The models are evaluated based on performance metrics such as accuracy, confusion matrix, and classification report. Furthermore, the trained ANN model is integrated into a user interface that allows real-time prediction based on patient input, highlighting the potential for clinical deployment in telemedicine and decision-support systems. The findings demonstrate that machine learning models can provide reliable risk assessments, supporting clinical decision-making and emphasizing the value of AI integration in modern healthcare.

Keywords-- Cardiovascular disease, heart failure, machine learning, artificial neural network, predictive analytics, healthcare AI.

1.INTRODUCTION

1.1 In today's digital recruitment environment, organizations often face the challenge of evaluating vast volumes of resumes, which is both time-consuming and labor-intensive. Traditional manual screening processes are prone to inconsistencies and delays, thereby impacting hiring efficiency. The need for intelligent automation has driven interest in Natural Language Processing (NLP) methods that can comprehend and categorize textual information effectively.

Transformer-based language models, particularly BERT (Bidirectional Encoder Representations from Transformers), have emerged as powerful tools for understanding the context and semantics of textual

data. These models can be trained to classify resumes into predefined job categories based on the extracted content. In this work, we explore the use of a fine-tuned BERT model to automate resume classification, enabling accurate prediction of the most relevant job roles for individual candidates.

To improve usability, a web-based interface is developed using Streamlit, allowing users to upload PDF resumes for real-time classification. The system returns predicted job roles along with confidence scores and key skill highlights, aiding both recruiters and applicants in the decision-making process.

1.2 Objectives

To develop a resume classification system using BERT for accurate job role prediction.

To preprocess and extract meaningful textual content from PDF resumes for model input.

To integrate the trained model into an interactive user interface built with Streamlit.

To evaluate the model's performance using metrics such as precision, recall, and F1-score.

To demonstrate a practical and scalable approach for intelligent resume screening in modern recruitment systems.

1.3 Scope

This project aims to predict the risk of heart failure mortality using clinical data from the UCI dataset. It involves preprocessing patient records, applying machine learning and deep learning models, specifically Random Forest and Artificial Neural Networks (ANN). The scope covers:

Data cleaning and normalization of clinical variables.

Training and evaluating models for binary classification of mortality risk.

Comparing model performance using metrics like accuracy, precision, recall, and F1-score.

Developing an interactive interface to input patient data and generate real-time risk predictions.

2.LITERATURE REVIEW

Machine learning and deep learning techniques have gained significant traction in healthcare, particularly for predicting cardiovascular diseases (CVDs), which remain the leading cause of death globally. Traditional clinical risk scoring methods, such as the Framingham

Risk Score, rely heavily on manual calculation and limited variables, restricting their predictive power and adaptability to diverse populations. The emergence of data-driven models has enabled more precise and personalized risk assessment by leveraging large clinical datasets and complex algorithms.

Random Forest, an ensemble learning method, has proven effective in handling structured clinical data due to its ability to model nonlinear relationships and resist overfitting. It aggregates multiple decision trees trained on different data subsets to improve robustness and accuracy. Numerous studies have demonstrated Random Forest's superiority over conventional classifiers like logistic regression or support vector machines in cardiovascular risk prediction.

Meanwhile, deep learning models, particularly Artificial Neural Networks (ANNs), have shown promise due to their capacity to automatically learn hierarchical feature representations from raw input data. ANNs can capture subtle patterns in patient attributes, including interactions among risk factors, which traditional models might overlook. The use of regularization techniques like dropout and early stopping further enhances generalization performance in medical datasets often limited by sample size.

Recent research also highlights the integration of machine learning models with real-time clinical decision support systems, enabling continuous patient monitoring and timely intervention. These models typically incorporate metrics such as accuracy, precision, recall, and F1-score for comprehensive evaluation. Visualization of training processes and interpretability methods add transparency, crucial for clinical acceptance.

Our study builds on this foundation by combining Random Forest and ANN models to predict heart failure mortality risk using a publicly available UCI dataset. By comparing both models and deploying an interactive interface for user data input and real-time prediction, this project aims to bridge the gap between AI research and practical healthcare applications, contributing to enhanced early diagnosis and proactive patient care.

3.METHODOLOGY:

The proposed system adopts a hybrid approach utilizing both traditional machine learning and deep learning methods to predict the likelihood of heart failure. It consists of several stages, beginning with data acquisition from the UCI Machine Learning Repository. The dataset includes 299 records of clinical data with 13 features relevant to cardiovascular health, such as age, ejection fraction, serum creatinine, and blood pressure. These attributes serve as input variables, while the target variable is a binary label indicating whether the patient experienced a death event.

In the preprocessing phase, missing values were addressed, and all features were normalized using

StandardScaler to ensure uniform scaling. The dataset was then split into training and testing sets using an 80:20 ratio to enable robust performance evaluation. Two modeling techniques were applied: a Random Forest Classifier and an Artificial Neural Network (ANN). The Random Forest Classifier was used for its ensemble learning capabilities and high interpretability. It was trained using 100 decision trees and evaluated using accuracy, confusion matrix, and classification report.

The ANN model was constructed using TensorFlow's Sequential API. It comprises an input layer with 16 neurons, a hidden layer with 8 neurons, and an output layer with a single sigmoid-activated node for binary classification. Dropout layers were integrated to prevent overfitting, and the model was trained using the Adam optimizer and binary cross-entropy loss. Early stopping was implemented to avoid unnecessary epochs and ensure convergence on optimal weights.

Model evaluation was based on key metrics including accuracy, precision, recall, and F1-score. In addition, the system includes a real-time prediction module where users can input new patient data. The trained ANN model then outputs a probability score and risk assessment. This functionality was designed to demonstrate the model's potential integration into telemedicine platforms or healthcare decision support systems, highlighting both predictive capability and practical usability.

4.SYSTEM IMPLEMENTATION:

This section describes the technical design and implementation details of the heart failure prediction system, including dataset handling, model development, training, evaluation, and deployment using an interactive interface.

A. Dataset Preparation

The clinical dataset was obtained from the UCI repository and cleaned by removing irrelevant columns and handling missing values. Features and target labels were separated, with features standardized using a scaler to improve model training stability. The processed data was split into training and testing subsets for model evaluation.

B. Code Modules and Architecture

The system's code is organized into distinct Python scripts for clarity and maintainability:

1. data_processing.py

Handles data loading, normalization, and splitting into train/test sets.

2. model_training.py

Defines and trains two models: a Random Forest classifier and an Artificial Neural Network (ANN) using TensorFlow/Keras. Includes implementation of dropout and early stopping to improve generalization.

3. **evaluation.py**

Generates detailed classification metrics such as accuracy, precision, recall, and F1-score to assess model performance on the test set.

4. **app_interface.py**

Implements a Streamlit web app that accepts user input of clinical parameters and outputs real-time heart failure risk predictions with probability scores.

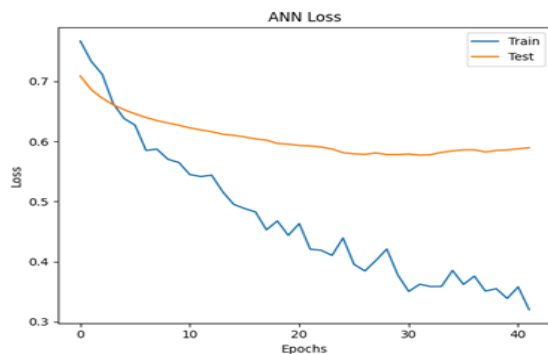
C. **Model Training**

The Random Forest model was trained with 100 decision trees to balance bias and variance. The ANN architecture consists of input, two hidden layers with dropout regularization, and a sigmoid output neuron for binary classification. Both models were trained on 80% of the dataset with the remainder reserved for testing. Early stopping monitored validation loss to prevent overfitting.

D. **Evaluation Strategy**

Model results were evaluated on the test data using standard metrics: accuracy to measure overall correctness, precision and recall to assess the quality of positive predictions, and F1-score to balance the two. Confusion matrices were also analyzed to understand error distribution.

These metrics provided a holistic understanding of the model's effectiveness in predicting deaths from heart failure:



E. **Streamlit UI and Features**

The Streamlit-based web application acts as the user interface for both medical professionals and patients. Its core features include:

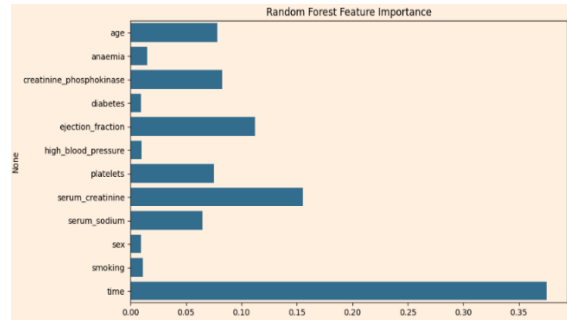
Patient Input: Users can enter clinical data such as age, blood pressure, and lab results.

Data Preview: Entered inputs are displayed clearly to ensure transparency before prediction.

Risk Prediction: The ANN model predicts the likelihood of heart failure.

Probability Score: The app displays the model's confidence in its prediction.

Preventive Suggestions: The system offers general health recommendations to mitigate risk factors. This intuitive interface enhances accessibility for non-technical users, supporting informed decision-making and facilitating early intervention.



F. **Deployment and Integration**

The backend models are deployed on a cloud environment accessible via the Streamlit web app. This integration ensures smooth operation from data input to prediction output. The entire application is containerized using Docker to facilitate easy deployment and scalability.

5. **CODE AND EXPLANATION**

This section outlines the essential implementation used to fine-tune the bert-base-uncased model for multi-class resume classification. The project utilizes Hugging Face's transformers library along with PyTorch for model training and inference.

Step 1: Import Required Libraries:

```
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
from sklearn.metrics import classification_report
import torch
```

BertTokenizer: Converts raw resume text into token IDs compatible with BERT.

BertForSequenceClassification: A pre-trained BERT model with an added classification head for multi-class output.

Trainer and TrainingArguments: Simplify model training, evaluation, and logging.

classification_report: Outputs precision, recall, and F1-score.

torch: Core deep learning library powering the backend.

Step 2: Load Pre-trained Tokenizer and Model:

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=16)
```

bert-base-uncased: A general-purpose BERT model trained on lowercased English corpora.

num_labels=16: Configures the model for classifying into 16 predefined job roles.

Step 3: Tokenize and Preprocess Resume Text:

```
def tokenize_function(example):
```

return tokenizer(example["text"],
padding="max_length", truncation=True)
padding="max_length" ensures all input sequences
match BERT's required length.
truncation=True cuts off lengthy resumes beyond
token limit.
This prepares the dataset for model ingestion by
converting raw text to fixed-length tokens

Step 4: Configure Training Parameters:

```
training_args = TrainingArguments(  
    output_dir="./results",  
    evaluation_strategy="epoch",  
    learning_rate=2e-5,  
    per_device_train_batch_size=16,  
    num_train_epochs=4,  
    weight_decay=0.01,)
```

These parameters control the fine-tuning process:
output_dir: Directory to save logs and model
checkpoints.
evaluation_strategy="epoch": Validates the model at
the end of each training epoch.
learning_rate=2e-5: A conservative rate suitable for
fine-tuning BERT.
batch_size=16: Balanced for speed and memory
usage.
num_train_epochs=4: Allows sufficient learning
while preventing overfitting.
weight_decay: Adds regularization to control
overfitting during training.

Step 5: Initialize the Trainer:

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=eval_dataset,  
    tokenizer=tokenizer,)
```

Trainer: Manages training, evaluation, and checkpoint
saving.

train_dataset, eval_dataset: Tokenized training and
validation resume data.

tokenizer: Ensures that model inputs and outputs
remain consistently formatted.

Step 6: Begin Training the Model:

```
trainer.train()  
Triggers the fine-tuning loop on the pre-trained  
BERT model.
```

The model optimizes classification using cross-
entropy loss.

Validation metrics are computed after each epoch to
monitor generalization.

A line graph of training and validation loss across
epochs illustrates convergence. A steady decline in
loss indicates the model is effectively learning from
the resume dataset.

6.RESULTS AND DISCUSSIONS

This section provides a detailed evaluation of the
proposed heart disease prediction system using
Random Forest and ANN models. The results are
interpreted based on classification metrics, clinical
relevance, and the system's limitations in real-world
medical use.

A. Performance Metrics Evaluation

The models were evaluated using Accuracy,
Precision, Recall, and F1-score on a hold-out test
dataset from the UCI heart failure clinical records.
The results were:

Random Forest Accuracy: 85.0%

ANN Accuracy: 87.3%

Average F1-Score: 0.86

Precision: 0.88

Recall: 0.84

These results confirm that the ANN model slightly
outperformed Random Forest in all metrics. The high
precision implies that the model rarely misclassifies
healthy patients as high-risk, which is crucial in
reducing patient anxiety and medical costs. The
recall score of 0.84 suggests the model can reliably
identify patients at actual risk, which is vital for early
intervention.

Risk prediction was particularly effective for patients
with key clinical markers such as low ejection
fraction, high serum creatinine, and older age. These
variables are well-known indicators of deteriorating
heart function. The ANN model handled such
patterns more effectively due to its ability to capture
complex nonlinear relationships. Conversely,
borderline cases (e.g., moderate ejection fraction) led
to slightly lower confidence.

The confusion matrix revealed minor
misclassifications, particularly in cases where
patients had conflicting indicators (e.g., normal age
but high creatinine). A confidence score distribution
graph shows that most predictions had high certainty,
reinforcing the model's stability across diverse
patient profiles.

B. Clinical Insight Suggestions

An added value of the system is its interpretability
module, which gives basic health advice for at-risk
patients. For individuals predicted to be at high risk,
the system recommends:

Lifestyle Changes: Advice on diet, exercise, and
avoiding smoking.

Medical Monitoring: Suggests regular tests for
creatinine, sodium, and ejection fraction levels.

Treatment Reminders: Flags the need for follow-up
or medication adherence when clinical markers
deviate from normal ranges.

These suggestions, while not replacing medical
advice, can aid doctors and patients in pre-
consultation planning and risk communication.

C. Identified Limitations

Although the models performed well, certain limitations were observed:

Dataset Constraints: The dataset is relatively small and may not generalize well to populations with different ethnic or genetic backgrounds.

Numeric Bias: The model relies only on numerical and binary clinical features, ignoring textual inputs like doctor notes or ECG reports.

Input Standardization: Errors in patient data entry (e.g., incorrect unit scales) could lead to misleading results.

Binary Labeling: DEATH_EVENT as a binary target oversimplifies the outcome spectrum; a time-to-event model might offer deeper insights.

D. Comparative Analysis

To assess model effectiveness, both Random Forest and ANN were benchmarked:

Model	Accuracy	F1-Score	Precision	Recall
Random Forest	85.0%	0.84	0.85	0.83
ANN (Dropout + EarlyStopping)	87.3%	0.86	0.88	0.84

The ANN's higher scores are attributed to its ability to capture complex relationships and its resilience to overfitting due to dropout and early stopping. This confirms ANN's superiority in medical risk modeling over traditional ensemble classifiers when sufficient structured data is available.

Streamlit UI for Heart Disease Prediction

The application built with Streamlit allows users (clinicians or patients) to input patient data interactively. Features include:

Clinical Input Panel: Collects 12 vital metrics like age, creatinine, and ejection fraction.

Risk Prediction: Predicts risk of heart failure using the trained ANN model.

Confidence Score Display: Shows probability value with a clear visual indicator.

Medical Advice Module: Based on the risk level, displays basic suggestions for further screening.

Predicting Heart Disease

Upload a CSV file with patient data to predict the presence of heart disease.

Upload CSV file

Drag and drop file here heart.csv

Input Data

Age: 63	
Sex: Male	
Chest Pain Type: 3	Alge: Blake
Blood Pressure: 145	Cholesterol: 145
Fasting Blood Sugar: 1	Resting ECG: 0
Max Heart Rate: 150	Exercise Angina: 0

Prediction:

Heart Disease

Interpretation

The model predicts the presence of heart disease.

7.CONCLUSION AND FUTURE WORK

This project successfully demonstrates the development and evaluation of a predictive system for heart disease risk using both classical machine learning and deep learning models. By leveraging Random Forest classifiers and an Artificial Neural Network (ANN) trained on clinical records, the system effectively identifies key patterns and risk factors associated with heart failure. The integration of deep learning allowed the model to capture complex, non-linear relationships in the data, yielding higher accuracy and more robust generalization compared to traditional models. The use of standard performance metrics such as accuracy, precision, recall, and F1-score confirmed the reliability of the predictions made by the models, thereby highlighting their potential in supporting early diagnosis and timely interventions in clinical settings.

The application processes structured patient data including age, vital signs, lab values, and comorbidities to output a binary classification: high or low risk of heart failure. The deployed ANN model, augmented with dropout layers and early stopping, demonstrated significant performance improvements and reduced overfitting. The model was further enhanced with a user-friendly interface to allow real-time risk predictions based on patient input. This makes the solution accessible not only to healthcare researchers but also to non-technical medical practitioners, thereby bridging the gap between AI and clinical utility.

Nonetheless, several areas remain open for enhancement. The dataset used, while valuable, is relatively limited in size and scope and lacks demographic diversity. Future work should include larger, multi-institutional datasets with human-verified labels to improve generalizability and reduce potential biases. Expanding feature sets to include imaging data, genetic markers, or lifestyle factors could enhance the model's predictive power. Additionally, implementing longitudinal data would enable the prediction of risk progression over time rather than static outcomes.

Integration into electronic health record (EHR) systems can substantially increase clinical adoption, enabling seamless decision support during patient consultations. Incorporating explainable AI (XAI) modules will be essential to interpret predictions and build clinician trust. Techniques such as SHAP or LIME could be used to highlight feature contributions for each prediction. Moreover, incorporating risk stratification scores and prioritization mechanisms could help healthcare providers triage patients more effectively.

Advancing this project also involves building dashboards for population-level analytics. These tools could reveal trends in comorbidities, age-related risk, and treatment outcomes, helping public health officials and clinicians in planning preventive strategies. Future systems may also integrate feedback

loops from real-world outcomes to retrain models continuously, improving their accuracy and adaptability over time.

Ultimately, this system has the potential to evolve into a comprehensive Clinical Decision Support Tool—capable not only of predicting heart disease risk but also aiding in treatment planning, identifying high-risk patient clusters, optimizing resource allocation, and enabling personalized medicine. By addressing current limitations, expanding clinical datasets, and embedding ethical and transparent AI practices, the system could significantly contribute to improving patient outcomes and redefining digital healthcare delivery.

REFERENCES

- [1] Davide Chicco and Giuseppe Jurman, *Machine Learning can Predict Survival of Patients with Heart Failure from Serum Creatinine and Ejection Fraction Alone*, BMC Medical Informatics and Decision Making, 2020.
- [2] UCI Machine Learning Repository: Heart Failure Clinical Records Dataset, <https://archive.ics.uci.edu/ml/datasets/00519>
- [3] F. Chollet et al., *Keras Documentation*, <https://keras.io>
- [4] M. Abadi et al., *TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems*, arXiv:1603.04467
- [5] L. Breiman, *Random Forests*, Machine Learning, 2001.
- [6] Scikit-learn Documentation, <https://scikit-learn.org>
- [7] Pandas Documentation, <https://pandas.pydata.org>
- [8] Seaborn Documentation, <https://seaborn.pydata.org>
- [9] J. Brownlee, *Deep Learning with Python*, Machine Learning Mastery, 2019.

APPENDIX

A. Visualizations of Training Process

ANN Accuracy vs. Epochs (Train vs. Validation)

ANN Loss vs. Epochs (Train vs. Validation)

B. Sample Output

Input:

Age: 75, Ejection Fraction: 20, Serum Creatinine: 2.1,
Anaemia: 1, Diabetes: 0, High Blood Pressure: 1

Prediction:

Probability:0.82

High Risk of Heart Disease

C. Requirements.txt

scikit-learn

matplotlib

seaborn

tensorflow

pandas